

## **PROYECTO: Controlador SPI en Verilog**



Materia: Arquitectura de Computadoras

Profesor: JORGE ERNESTO LOPEZ ARCE DELGADO

Integrantes: Diego Israel Gonzalez Sanchez

Derek Gabriel Casillas Lopez

Iván Samuel Ortiz Orellana

Omar Luna Reyes

Fecha: 06/03/2026

# 1. AVANCE TÉCNICO (Verilog y Lógica)

## 1.1 Definición de Módulos (Verilog)

```
module spi_master(  
  
    input wire clk,          // Reloj del sistema  
  
    input wire rst,          // Reset  
  
    input wire start,        // Señal para iniciar transmisión  
  
    input wire [7:0] data_in, // Dato a enviar  
  
    output reg [7:0] data_out, // Dato recibido  
  
    output reg sclk,         // Serial Clock  
  
    output reg mosi,         // Master Out Slave In  
  
    input wire miso,         // Master In Slave Out  
  
    output reg ss,           // Slave Select  
  
    output reg done          // Señal de transmisión completa  
  
);  
  
// Estados de la FSM  
  
parameter IDLE = 2'b00;  
  
parameter SETUP = 2'b01;  
  
parameter TRANSMIT = 2'b10;  
  
parameter DONE = 2'b11;  
  
reg [1:0] state;  
  
reg [2:0] bit_counter;  
  
reg [7:0] shift_reg;  
  
always @(posedge clk or posedge rst) begin  
  
    if (rst) begin  
  
        state <= IDLE;  
  
        ss <= 1;  
  

```



```

        end

    end

    DONE: begin

        ss <= 1;

        done <= 1;

        state <= IDLE;

    end

endcase

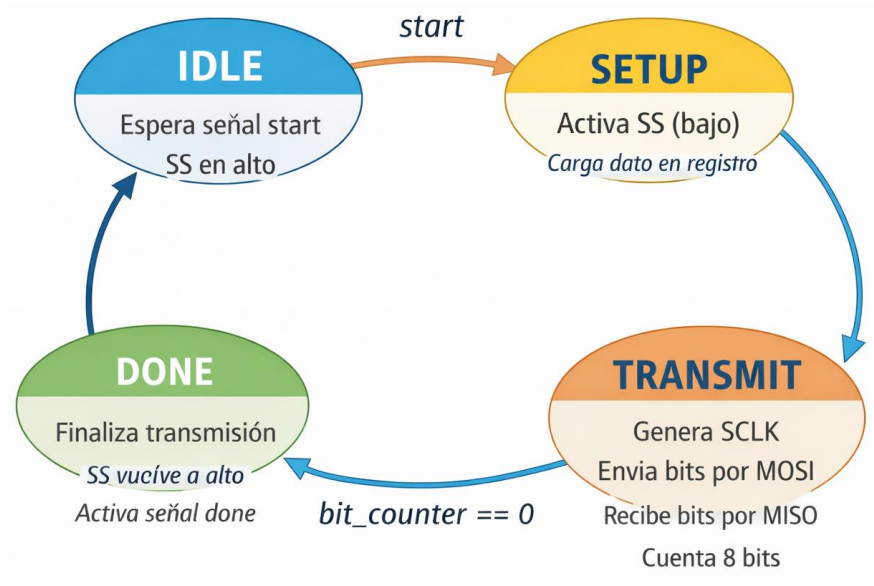
end

end

endmodule

```

## 1.2 Diagrama de la Máquina de Estados Finitos (FSM)



## 1.3 Plan de Pruebas (Testbench)

### 1. Transmisión de un byte

Se envía un dato de prueba (ej. 10101011).

**Resultado esperado:**

- SS en bajo durante transmisión.
- 8 pulsos de SCLK.
- MOSI transmite correctamente los bits.
- FSM: IDLE → SETUP → TRANSMIT → DONE → IDLE.

**2. Verificación de reloj (SCLK)**

Se observa la señal de reloj durante la transmisión.

**Resultado esperado:**

- SCLK activo solo cuando SS está en bajo.
- 8 ciclos completos por cada byte.

**3. Recepción por MISO**

Se simula un dato de respuesta del esclavo.

**Resultado esperado:**

- El dato recibido coincide con el enviado por el esclavo.
- La FSM pasa correctamente a DONE al finalizar.

## **2. AVANCE DE REPORTE TÉCNICO**

### **2.1 Introducción**

En esta práctica se desarrolla e implementa un controlador del protocolo Serial Peripheral Interface (SPI) utilizando el lenguaje de descripción de hardware Verilog. La actividad se centra en comprender el funcionamiento interno de los protocolos de comunicación serial síncrona y en aplicar los conceptos teóricos de diseño digital, máquinas de estados finitos y transferencia de datos serial en un entorno de simulación.

Durante la investigación previa se analizaron los fundamentos del protocolo SPI, su estructura maestro-esclavo, las líneas de comunicación que lo componen (SCLK, MOSI, MISO y SS), así como sus diferentes modos de operación determinados por los parámetros CPOL (Clock Polarity) y CPHA (Clock Phase). También se revisaron conceptos clave como registros de desplazamiento, sincronización por flanco de reloj, transmisión full-duplex y control secuencial mediante máquinas de estados finitos (FSM).

El protocolo SPI fue introducido originalmente por la empresa Motorola en la década de 1980 como una solución de comunicación de alta velocidad entre microcontroladores y dispositivos periféricos. A diferencia de otros protocolos como I2C o UART, SPI no utiliza direccionamiento interno ni confirmación automática, lo que simplifica su implementación hardware y permite mayores velocidades de transferencia.

### **2.2 Marco Teórico**

El diseño de sistemas digitales modernos se realiza mediante lenguajes de descripción de hardware (HDL) como Verilog. A diferencia de los lenguajes de programación tradicionales, Verilog permite describir comportamiento concurrente y estructuras físicas reales de hardware.

Según Mano y Ciletti, el diseño digital puede dividirse en:

- Diseño combinacional
- Diseño secuencial
- Diseño estructural
- Diseño conductual

En esta práctica se utiliza diseño secuencial, ya que el comportamiento depende del reloj y del estado previo del sistema.

## 2. Protocolo SPI

El **Serial Peripheral Interface** es un protocolo de comunicación serial síncrono que opera bajo una arquitectura Maestro-Esclavo.

### Características principales:

- Comunicación full-duplex
- Sin direccionamiento interno
- Alta velocidad
- Controlado por reloj
- Implementación simple en hardware

## 3. Líneas de Comunicación

SPI utiliza cuatro líneas principales:

1. **SCLK (Serial Clock)**  
Generado por el Maestro.
2. **MOSI (Master Out Slave In)**  
Línea de datos del Maestro al Esclavo.
3. **MISO (Master In Slave Out)**  
Línea de datos del Esclavo al Maestro.
4. **SS (Slave Select)**  
Habilita el esclavo cuando está en nivel bajo.

## 4. Modos de Operación (CPOL y CPHA)

El comportamiento del reloj depende de dos parámetros:

### CPOL (Clock Polarity)

Define el nivel base del reloj:

- CPOL = 0 → reposo en bajo
- CPOL = 1 → reposo en alto

### CPHA (Clock Phase)

Define en qué flanco se muestrea el dato:

- CPHA = 0 → primer flanco
- CPHA = 1 → segundo flanco

Esto genera cuatro modos posibles (0,1,2,3).

## 5. Registros de Desplazamiento

La transmisión SPI se basa en registros de desplazamiento. En cada pulso de reloj:

$$\begin{array}{l} \text{Shift\_Register} \leftarrow \text{Shift\_Register} \ll 1 \\ \text{Shift\_Register} \ll 1 \end{array} \quad \begin{array}{l} \text{Shift\_Register} \\ \text{Shift\_Register} \leftarrow \text{Shift\_Register} \ll 1 \end{array}$$

Mientras se transmite un bit por MOSI, simultáneamente se recibe uno por MISO.

## 6. Máquina de Estados Finitos (FSM)

Una Máquina de Estados Finitos es un modelo matemático que describe el comportamiento de un sistema secuencial.

Se define por:

- Conjunto de estados
- Entradas
- Salidas
- Función de transición

En esta práctica se implementan los estados:

- IDLE
- SETUP
- TRANSMIT
- DONE

La FSM permite controlar ordenadamente la activación de SS, la generación de SCLK y el conteo de bits transmitidos.

## 2.3 Bibliografía (APA)

Bhasker, J. (2005). *A Verilog HDL Primer* (3rd ed.). Star Galaxy Publishing.

Brown, S., & Vranesic, Z. (2014). *Fundamentals of Digital Logic with Verilog Design* (3rd ed.). McGraw-Hill Education.

Mano, M. M., & Ciletti, M. D. (2013). *Digital Design with an Introduction to Verilog HDL* (5th ed.). Pearson.



Wakerly, J. F. (2005). *Digital Design: Principles and Practices* (4th ed.). Pearson Prentice Hall.

Wolf, W. (2012). *Computers as Components: Principles of Embedded Computing System Design* (3rd ed.). Morgan Kaufmann.

## **2.4 Roles y Responsabilidades**

Derek Gabriel Casillas Lopez: Código Verilog.

Omar Luna Reyes: FSM.

Iavn Samuel Ortiz Orellana: Simulación.

Diego Israel Gonzalez Sanchez: Reporte/Presentacion.

## **3. AVANCE DE VIDEO**

### **3.1 Guion Literario**

#### **Introducción**

"En este video explicaremos brevemente qué es el protocolo de comunicación SPI, dónde se utiliza y cómo funciona dentro de los sistemas electrónicos digitales."

#### **¿Qué es SPI?**

"SPI, que significa Serial Peripheral Interface, es un protocolo de comunicación serial síncrona utilizado para conectar un microcontrolador con diferentes dispositivos periféricos. Este protocolo permite la transmisión de datos de manera rápida entre un dispositivo maestro y uno o varios dispositivos esclavos."

#### **¿Dónde se usa?**

"El protocolo SPI es ampliamente utilizado en sistemas embebidos y aplicaciones electrónicas modernas. Algunos ejemplos comunes incluyen memorias Flash, pantallas LCD, convertidores analógico-digitales, sensores y tarjetas SD. Su popularidad se debe a su alta velocidad de transferencia y su implementación relativamente sencilla."

## ¿Cómo funciona?

"En una comunicación SPI existe un dispositivo maestro que controla toda la comunicación. El maestro genera la señal de reloj conocida como SCLK y selecciona al dispositivo esclavo mediante la línea SS o Slave Select. Los datos se envían desde el maestro hacia el esclavo a través de la línea MOSI, mientras que el esclavo puede responder enviando datos al maestro por la línea MISO. Todo este proceso ocurre sincronizado con la señal de reloj."

## Estados del controlador

"Para implementar esta comunicación se diseñó un controlador SPI basado en una máquina de estados finitos. Este controlador incluye cuatro estados principales: IDLE, donde el sistema espera iniciar una transmisión; SETUP, donde se preparan las señales de comunicación; TRANSMIT, donde se realiza el envío de los datos; y DONE, que indica que la transmisión ha finalizado."

## Formato de trama

"Durante cada operación de comunicación se transmite una trama de datos compuesta por 8 bits. Cada bit es enviado y recibido de manera sincronizada con la señal de reloj generada por el dispositivo maestro."

## Cierre

"De esta manera, el protocolo SPI permite una comunicación rápida, eficiente y ampliamente utilizada en diversos sistemas electrónicos."

## 3.2 Storyboard

| Parte        | Visual                                                                      |
|--------------|-----------------------------------------------------------------------------|
| Introducción | Título animado del proyecto: "Protocolo de Comunicación SPI"                |
| ¿Qué es SPI? | Diagrama básico de comunicación Maestro – Esclavo                           |
| Líneas SPI   | Imagen señalando las líneas SCLK, MOSI, MISO y SS                           |
| FSM          | Diagrama de la máquina de estados (IDLE, SETUP, TRANSMIT, DONE)             |
| Código       | Fragmento del módulo SPI implementado en Verilog                            |
| Simulación   | Captura o video corto de la simulación en ModelSim mostrando la transmisión |
| Final        | Texto de conclusion del funcionamiento del protocolo SPI                    |

## 4. AVANCE DE PRESENTACIÓN

### 4.1 Estructura de Diapositivas

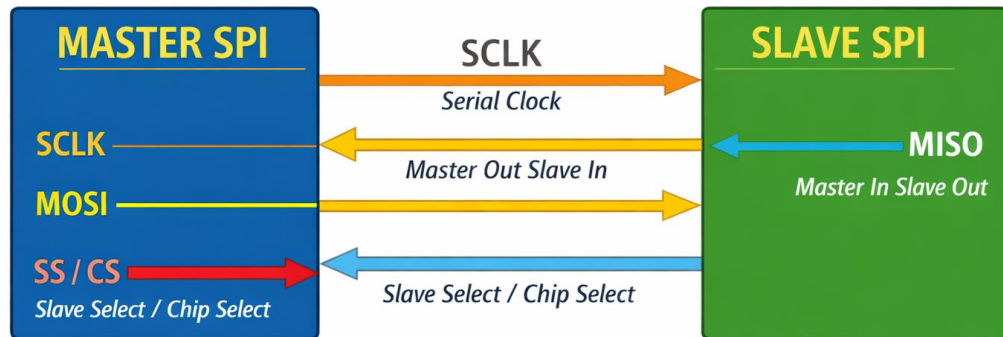
- **Portada**  
Se mostrará el título del proyecto, nombre del alumno, materia, profesor y fecha de presentación.
- **Objetivo**  
Se explicará el objetivo general del proyecto, que consiste en diseñar e implementar un controlador de comunicación SPI utilizando Verilog.
- **¿Qué es SPI?**  
Se presentará una explicación general del protocolo SPI, su propósito y sus características principales dentro de los sistemas digitales.
- **Arquitectura Maestro-Esclavo**  
Se mostrará el modelo de comunicación entre el dispositivo maestro y el esclavo, incluyendo las líneas de comunicación SCLK, MOSI, MISO y SS.
- **Modos CPOL/CPHA**  
Se explicarán los diferentes modos de operación del protocolo SPI, determinados por la polaridad del reloj (CPOL) y la fase de reloj (CPHA).
- **Diseño en Verilog**  
Se presentará la estructura del módulo desarrollado en Verilog, explicando las entradas, salidas y el funcionamiento general del controlador SPI.
- **FSM (Máquina de Estados Finitos)**  
Se mostrará el diagrama de estados utilizado para controlar la comunicación SPI, incluyendo los estados IDLE, SETUP, TRANSMIT y DONE.
- **Testbench**  
Se explicará el banco de pruebas utilizado para verificar el funcionamiento del módulo SPI mediante simulación.
- **Resultados**  
Se presentarán las simulaciones obtenidas en ModelSim que demuestran el correcto funcionamiento de la comunicación SPI.
- **Conclusiones**  
Se comentarán los resultados obtenidos, el aprendizaje adquirido durante la práctica y posibles mejoras del sistema.
- **Bibliografía**  
Se mostrarán las fuentes de información utilizadas para el desarrollo del proyecto.

## 4.2 Diagrama de Bloques del Sistema

Maestro genera SCLK y SS.

MOSI: Maestro → Esclavo.

MISO: Esclavo → Maestro.



## Enlace a carpeta de GitHub

[https://github.com/Diego-Gonzalez17/SPI\\_Controller\\_Verilog](https://github.com/Diego-Gonzalez17/SPI_Controller_Verilog)